TEXT2WEB · USER GUIDE

Build webpages by describing them in English

A practical guide to the supported instructions, component names, layouts, styles, and editing workflow in this project.

For the Text2Web compiler workspace
Offline English interpretation · HTML + CSS output

1. Get started

1. Start the local app from the project folder with `.\.venv\Scripts\python.exe app.py`.
2. Open `http://127.0.0.1:5000` in your browser.
3. Describe the complete page in the English instructions editor and choose **Compile Webpage**.
4. Review the live preview and generated HTML/CSS. Fix any compiler message, then compile again.
5. Use the follow-up editor and **Apply follow-up** to change the saved page without rewriting it.

> **Recommended pattern:** create the page first, then add structure and content, then arrange the layout, and finish with visual styling and responsive behavior.

A complete starter page

```
Create a page titled "Northstar Services".
Create a navigation bar with Home, About, Services and Contact links.
Create a hero section with a heading "Build what comes next", a description "Practical technology
services for growing teams." and a button "Explore services".
Create three cards for Web Development, AI and Cloud Computing.
Each card should have a short description "Talk to our team about your project." and a button
"Learn more".
Put all the cards inside one container.
Display three cards in a row on desktop.
Stack the cards vertically on mobile.
Use a clean, modern design with readable typography.
Make the layout responsive on mobile devices.
```

The quoted strings are the exact visible content. Replace them with your own title, labels, descriptions, and card names.

Instruction writing rules

- Use one instruction per line, or separate instructions with full stops.
- Put text, names, and titles in quotation marks. Double quotes are recommended; single quotes are also accepted.
- Keywords are not case-sensitive. Text inside quotes keeps its capitalization and punctuation.
- You can begin instructions with `Create`, `Add`, or `Insert`; `Please` is optional.
- Use explicit text and destinations. The compiler will not invent missing card descriptions, form options, or image details.

2. Page text and components

What you want	Example English instruction
Set the browser-tab title	`Create a page titled "Portfolio".`
Add a heading	`Add a heading that says "Welcome".`
Add a heading level 2–6	`Add a level 2 heading that says "Our services".`
Add a paragraph	`Add a paragraph that says "We build useful products."`
Add a visual button	`Add a button that says "Register".`
Add a semantic page section	`Create a section named "about".`
Add header, main content or footer	`Create a header named "site-header".`
Add a hero as one group	`Create a hero section with a heading "Welcome", a description "Explore our work." and a button "Learn more".`
Add separate named sections	`Create separate sections for About Us, Our Services and Contact.`

Supported structural components include headers, navigation bars, main content, sections, footers, sidebars, hero sections, containers and cards. A page may contain at most one explicit main section.

> **Button behavior:** buttons are clickable-looking controls with hover/pressed feedback, but they do not run an action or navigate. Use a link for navigation. Forms are static previews, not connected to a server.

Headings and content

A heading defaults to level 1. Specify a level for subheadings; heading levels are 1 through 6 and should not skip forward levels. Keep the page title and visible heading distinct: the title appears in the browser tab, while a heading appears on the page.

3. References, names, and containment

Name elements so follow-ups are clear

```
Create a card named "projects".
Add a heading that says "Projects" inside "projects".
Give "projects" 24 pixels of padding.
Make "projects" rounded.
```

Use `named "unique-name"` when creating a component, or assign a name later with `Set it name to "unique-name".` Names are case-sensitive, should be unique, and work well for follow-up edits.

Reference choices

- **By name:** `Make "projects" rounded.`
- **By exact text:** an exact visible text can identify an element if it is unique.
- **By type:** `Make the heading bold.` In normal English mode, if multiple elements share that type, specify which one.
- **By ordinal in a follow-up:** `Make the first card blue.` The first/second/third/last selector refers to elements of that type.
- **Pronoun:** `Make it rounded.` means the latest introduced/focused component when it is unambiguous; otherwise name the target.
- **All matching items:** use phrases such as `all cards` or `each card` where the instruction supports a group.

Put children inside their parent

```
Create a main named "content".
Create a section named "about" inside "content".
Add a heading that says "About us" inside "about".
Add a paragraph that says "We make useful tools." inside "about".
Put "about" inside "content".
```

Containment is explicit: creating a container does not automatically put later elements inside it. Use `inside` when creating a child, or say `Put "child" inside "parent".` Components must follow valid HTML parent-child rules; for example, list items go inside lists, and table cells go inside rows.

4. Cards and layout

Create titled cards

```
Create three cards for "Web Development", "AI" and "Cloud Computing".
Each card should have a short description "Talk to our team about your project." and a button
"Learn more".
Put all the cards inside one container.
Give all cards rounded corners, a subtle border and consistent spacing.
Add a subtle shadow to each card.
```

Card titles are used as their names, so refer to them as `"Web Development"`, `"AI"`, or `"Cloud Computing"`. You may also say `the first card` in follow-ups. Supply the actual description and button text; these are not made up automatically.

Grid, row, column, and responsive commands

Goal	Command
Group all cards into the shared grid	`Put all the cards inside one container.`
Three columns on a desktop screen	`Display three cards in a row on desktop.`
Stack cards on a small screen	`Stack the cards vertically on mobile.`
Equal-height cards	`Make the cards equal in height and align them neatly.`
Make a named container vertical	`Make "cards-area" a flexbox column.`
Make a named container horizontal	`Make "cards-area" a flexbox row.`
Use a grid layout	`Make "cards-area" a grid layout.`
Enable wrapping	`Make "cards-area" wrap.`
Responsive page rules	`Make the layout responsive on mobile devices.`
Responsive navigation	`Make the navigation bar adapt to smaller screens.`

“Vertically” means the components are stacked from top to bottom. Group cards in a container first when using the card grid commands. Three-column desktop layouts and mobile stacking are designed to work together.

5. Position and alignment

Align an item to its parent's left, center, or right

```
Add a container named "actions".
Add a button that says "Register" inside "actions".
Align the button to the left.
```

Use `Align the button to the left.`, `Align the button to the center.`, or `Align the button to the right.` You can target a named component too: `Align "register" to the right.` The equivalent form `Put "register" on the left.` is also supported. The element must exist and, for clear results, should be inside a container with available width.

Place one item relative to another

```
Add a heading that says "Stage".
Add a paragraph that says "Doors open at six."
Put the paragraph below the heading.
Add an image called "stage.png" with the description "Main stage".
Put the image to the left of the heading.
```

Supported relative directions: `left`, `right`, `above`, and `below`. Put/place/move can express these relations, for example `Place the image to the left of the heading.` Both referenced elements must share the same parent. The compiler builds row/column layouts and reports contradictory positioning instructions.

Align text and flex children

- `Center the heading.` centers text in the referenced element.
- `Make "intro" centered.` centers text in the named element.
- `Align the navigation horizontally to the center.` uses flex alignment.
- `Align the navigation vertically to the center.` aligns items on the other flex axis.

6. Navigation, links, images

Navigation and links

```
Create a navigation bar with Home, About, Services and Contact links.
Create a link labelled "View services" to "/services".
Create a button-style link to the registration page.
Open this external link in a new tab.
```

Home in the navigation macro points to `/`; other labels become lowercase hyphenated paths such as About Us → `/about-us`. Set an explicit destination when needed. Links accept labelled/labeled/called/that says plus a URL or path. Internal fragment links such as `#about` must point to a real ID in the page. A button-style link navigates; a plain button does not. New-tab external links use safe rel attributes.

Images need a source and description

```
Add an image called "logo.png" with the description "Company logo".
Set it name to "logo".
Display this image at 45 pixels wide and 60 pixels tall.
Make the image responsive and keep its aspect ratio.
Make the logo link to the homepage.
```

Use a safe local PNG, JPEG, GIF, or WebP path or an HTTP(S) URL. Images need alternative text unless explicitly decorative. Local images must be provided beside the source or uploaded in the web editor before export. Remote image URLs are not fetched or verified by the compiler. Responsive images keep their proportions and do not overflow their container.

7. Lists, tables, and forms

Lists

```
Add a numbered list of "Register", "Prepare your portfolio" and "Attend the interview".
Add an unordered list named "technologies".
Add a list item "Frontend" inside "technologies".
```

Say numbered/ordered for an ordered list or bulleted/unordered for an unordered list. Provide the real list items. To nest a list, place the inner list inside a list item.

Tables

```
Create a table with columns for Subject, Credits and Grade.
Add a row with "Compilers", "4" and "A".
```

Columns become header cells. Each table row must supply the same number of cells as the header.

Static form previews

```
Create a contact form with name, email and message fields.
Make the email field required.
Add a password field labelled "Password".
Add a checkbox field labelled "Subscribe to updates".
Add a dropdown for selecting a department with options Admissions, Support and Research.
Set the email maxlength to 254.
```

Contact/registration forms support name, email, message and password. Supported controls also include text/email/password/checkbox/radio inputs, textareas, selects, and submit buttons. Provide labels and explicit dropdown options. Forms are intentionally inert previews: submission, authentication, email, data storage, and databases are not connected; submit buttons are disabled.

8. Colors, typography, and visual styling

Color and typography

```
Set the page background-color to "light-grey".
Give the heading a text color of "dark-blue".
Make the paragraph bold.
Give the page a font family of "system-ui".
Center the heading.
```

Style area	Supported examples
Color names	blue, red, green, yellow, orange, purple, black, white, grey/gray, light-grey/light-gray, dark-blue, dark-green; also six-digit hex such as #2055c8.
Text size	small, medium, big/large; or a supported font-size such as 32 pixels.
Font family	Arial, serif, sans-serif, monospace, system-ui.
Font weight	normal, bold, 400, 500, 600, 700.
Text alignment	left, center, right, justify.
Text decoration	none, underline, line-through.

Color on text elements changes text color; a color on a button sets its background. Button and navigation foreground colors are selected for readability unless overridden. Low-contrast foreground/background combinations may fail accessibility validation.

Spacing, borders, effects, and transitions

```
Give "projects" 24 pixels of padding.
Give "projects" 16 pixels of gap.
Give "projects" rounded corners.
Give "projects" a corner radius of 12 pixels.
Give all cards rounded corners, a subtle border and consistent spacing.
Add a subtle shadow to "projects".
Give the button a hover background color of "dark-blue".
```

Common styles include width/height, padding, margin, gap, border radius, border width/color/style, subtle shadow, smooth transition, and supported hover effects. Dimensions accept px/rem/em/%/vw/vh and a limited set of auto values. Arbitrary CSS expressions and inline styles are deliberately rejected.

Style the page or a component

Natural phrases such as `Set <target> <property> to <value>`, `Change ... to ...`, and `Give ... a padding of ...` can set a supported property. Property names and values are checked against a whitelist; a diagnostic explains unsupported or invalid choices.

9. Writing Assistant and follow-ups

Use the Writing Assistant panel

1. Type or select an instruction in either editor.
2. Review the suggested supported wording in the Writing Assistant panel.
3. Select **Apply suggestion** to change that instruction, or use a builder action such as Page, Heading, Button, Cards, or Background.
4. Choose **Insert instruction** or **Replace instruction**, review the text, then compile.

Suggestions do not change your text automatically. The assistant is guidance for supported patterns, not an unrestricted chatbot or spell-checker.

Make changes with follow-up instructions

```
Make the first card blue.
Give "AI" 24 pixels of padding.
Make the navigation bar rounded.
Add a paragraph that says "Contact us to discuss your project." inside "content".
Align the button to the center.
```

Follow-up instructions act on the most recently saved successful page. Use explicit names or ordinals when there are multiple headings, cards, links, or buttons. If a follow-up fails, the previous successful saved page is preserved.

Useful short edits

- `Make it bold.` — emphasize the current, unambiguous component.
- `Make "projects" rounded.` — round a named card/container.
- `Change the navbar background to dark green.` — change navigation background.
- `Make the image responsive.` — keep the image within its layout.
- `Make the layout responsive on mobile devices.` — apply the responsive page layout.

10. Troubleshooting and limits

Problem	What to do
“Cannot resolve” a component	Create it first; then use its exact quoted name or unique visible text.
Several elements have the same type	Name the target, or in a follow-up use first/second/third/last.
A placement instruction fails	Ensure both elements exist and share the same parent; use a container for a group.
Card layout doesn't group cards	Say `Put all the cards inside one container.` before grid or mobile-stack commands.
Image or export fails	Check that the path is safe, file exists/is uploaded, and image has a description.
Text/color is rejected	Use a supported color/property and a readable foreground/background pair.
Button doesn't do anything	Expected: plain buttons have no action. Create a link/button-style link for navigation.
Form won't submit	Expected: forms are static previews; backend behavior is not part of this compiler.
English phrase isn't recognized	Use a command from this guide or the sentence guide. Keep requirements explicit and try one instruction per line.

What this project deliberately does not do

- It is not a full unrestricted English-to-website AI; unsupported phrases may need rewording.
- It does not generate arbitrary JavaScript, CSS, or HTML from instructions.
- Plain buttons have no application behavior; forms do not connect to a backend.
- External websites, URLs, and remote image availability are not checked for reachability.
- Accessibility checks cover selected issues but are not a complete WCAG certification.

> **Best results:** build a complete page from specific instructions, name important components, group elements before applying layout, and use one clear change at a time when refining it.

This guide describes the Text2Web English instructions implemented in this project. The compiler performs the final parsing, reference, property, layout, and accessibility checks.